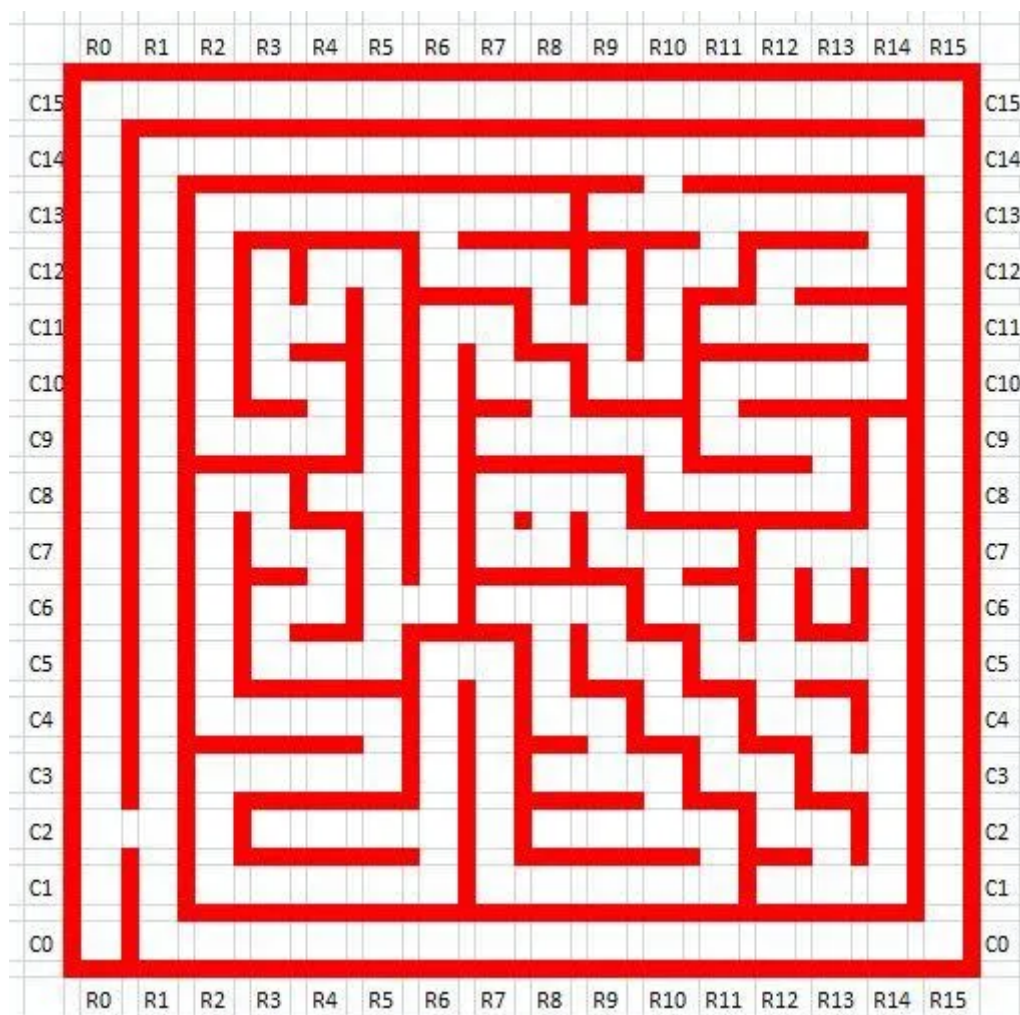


Micromouse from scratch| Algorithm- Maze traversal|Shortest path|Floodfill

How can your mouse find it's way in the big and insanely complicated maze.

The micromouse robot, as per the international [micromouse competition regulations](#), should be completely autonomous, should traverse the maze by itself while discovering the walls and mapping them in the memory. At the time of placing the robot on the maze, only the starting position and the destination position with respect to the starting position is known by the robot. Shown below is an example maze used in standard micromouse competitions.



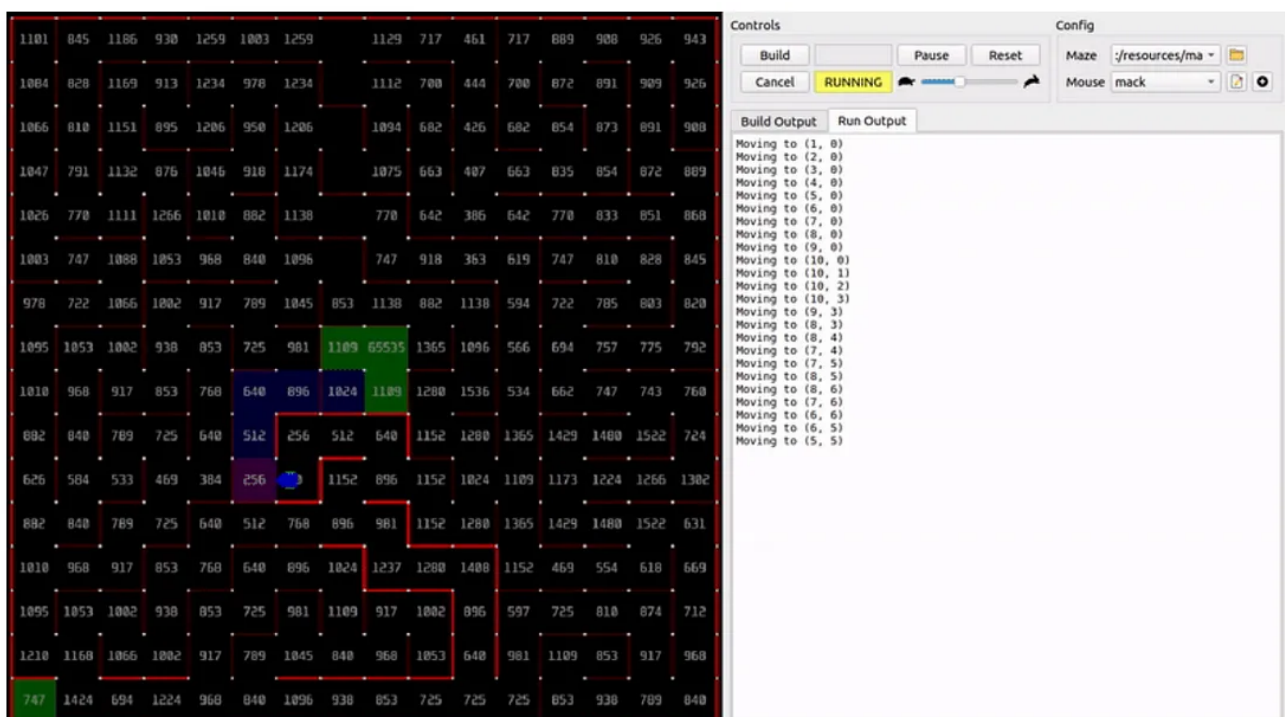
The starting position is at one of the four corners of the maze. The only requirement is that the starting square is enclosed by three walls to the left, right and back of the mouse. (In the above example, C0R0 qualifies as a starting square). The destination is at one of the four squares at the very center of the maze; (C7R7, C7R8, C8R7 and C8R8). The square enclosing the four destination positions can be accessed through one opening only. Reaching any one of the four squares counts as a successful run.

In this article, I will describe in detail the algorithm used by PICCOLA to find its way through the maze to the destination square, and back to the starting position, to and find the optimum/ shortest path to do the fast run.

The complete code used in our project can be found here in our [github repository](#).

A micromouse simulator

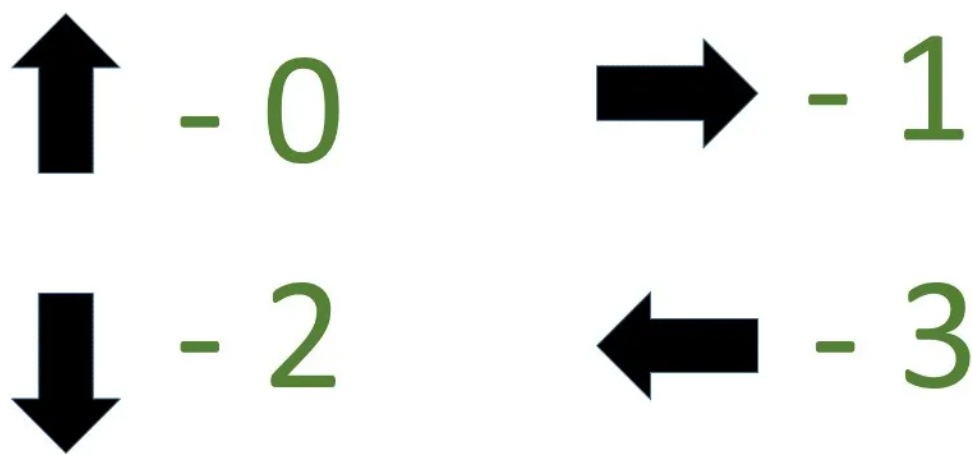
While I was developing and debugging the algorithm, it was hard to check it on the robot itself. So I installed a [micromouse simulator](#) I found on Github to see how a virtual mouse reacts to my algorithm. I set up the [python version](#), and made a few adjustments to the API.py file to configure some extra features for the simulator. (The files are available in [our repository](#))



Getting started with the algorithm

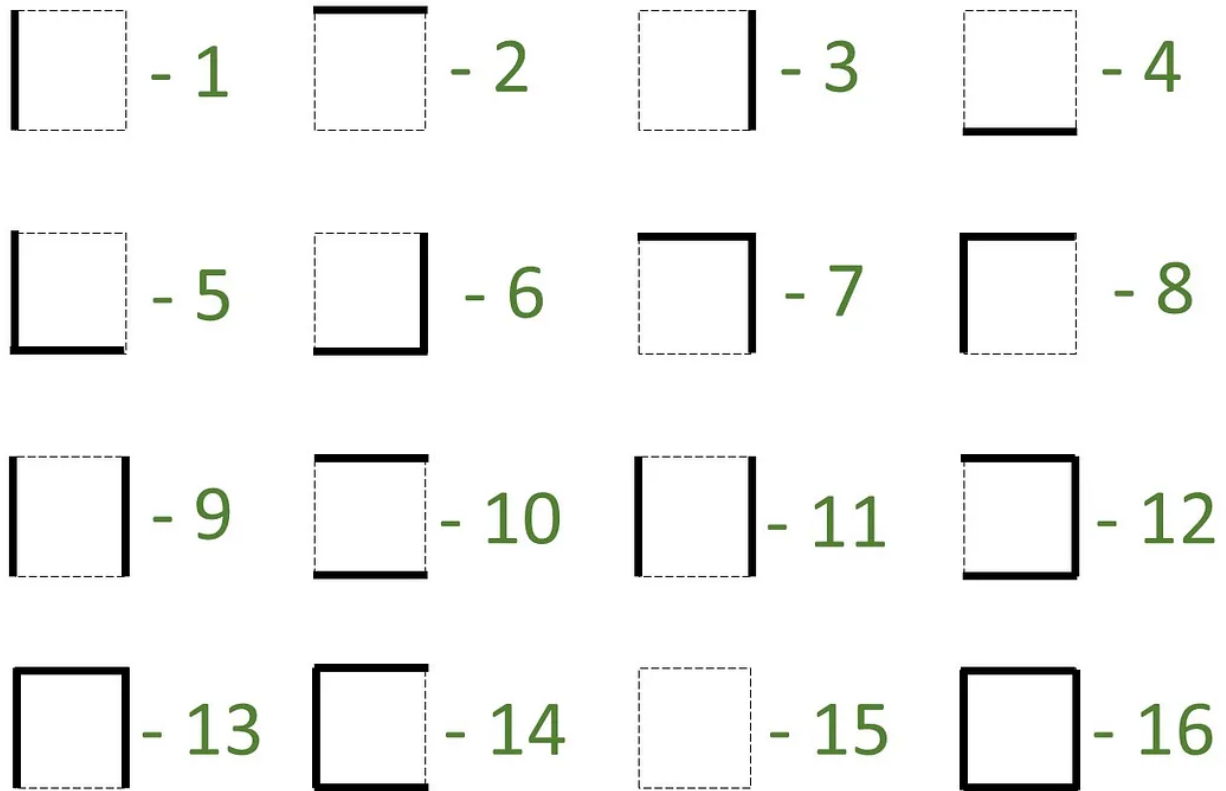
We represented the maze as a 2D array of size 16*16, initiated with zeros. We'll call it the **maze array**.

While the robot is in the maze, it can moving while facing towards four directions (orientations). To represent this in the algorithm, we used a variable **Orient** to store one of the four values; 0,1,2,3. Every time the robot performs a turn, the **orient** variable is updated.



'Orient' variable

Next we assigned different numbers to different configurations of walls a square can have. There can be 16 different combinations of walls (including a square with all four walls, and with no walls at all).



The numbers given to different wall configurations

As the mouse traverses through the maze, it can detect if a walls present to the left, right or in front of it, with the use of it's distance sensors. Next, depending on the availability of a wall to the left, right or front of the mouse, and the mouse's orientation while in the cell, we updated the corresponding entry in the **maze array** by its wall configuration.

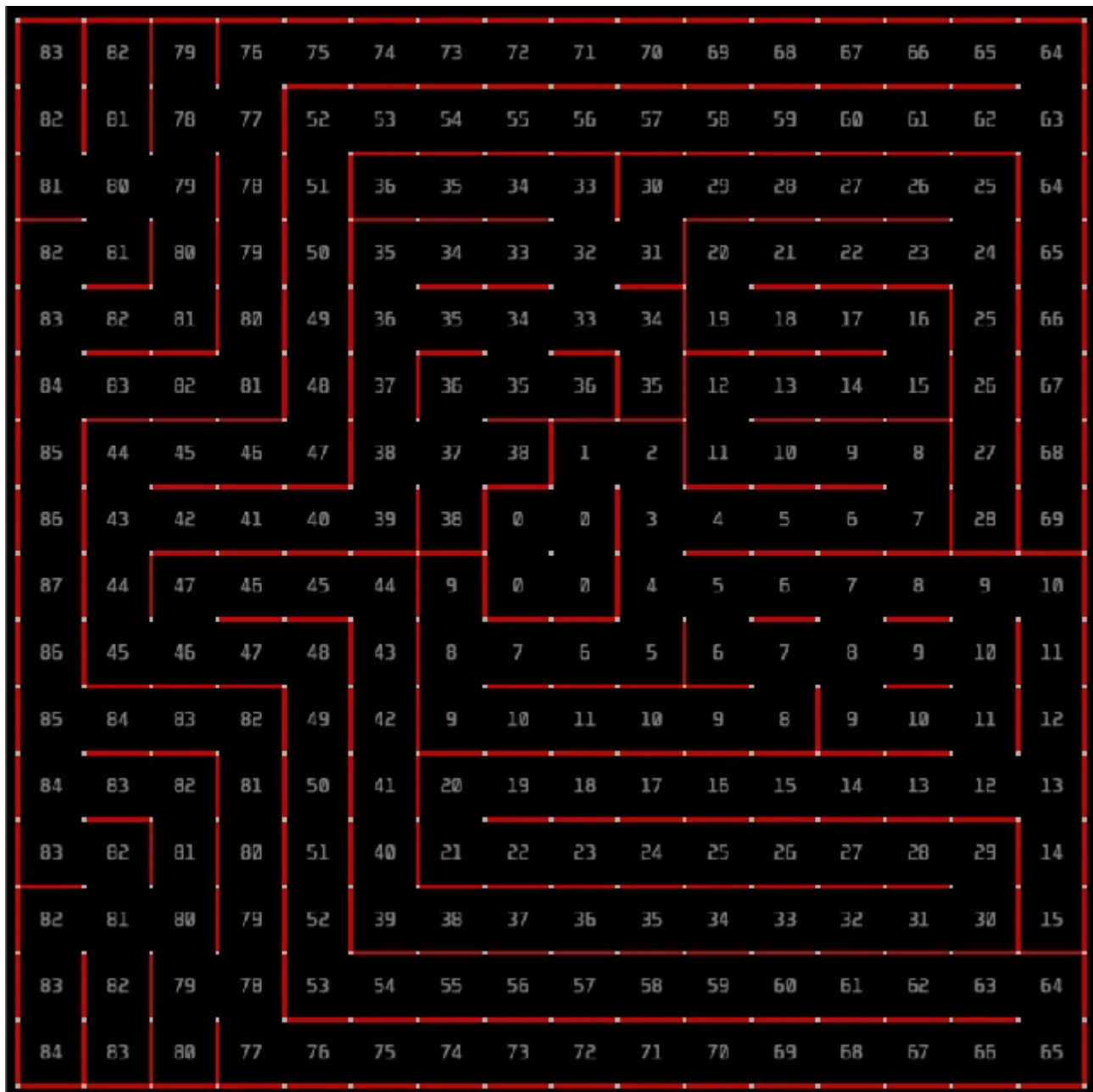
These 16 combinations were also categorized according to the presence of a wall to the left, right, up and down in a square.

The FloodFill algorithm

Imagine you pour water into the destination of the maze(which is the four center cells surrounded by 7 walls). The water will first flow to the cell immediately outside the destination cells. And then to it's immediately accessible neighboring cells. Similarly, water will flow along the paths in the maze, eventually reaching the starting position of the mouse.

We define another 16*16 maze, lets call it the **Flood array**. Inspired by the water pouring example, we assigned zero to the four destination cells, 1 to the cell

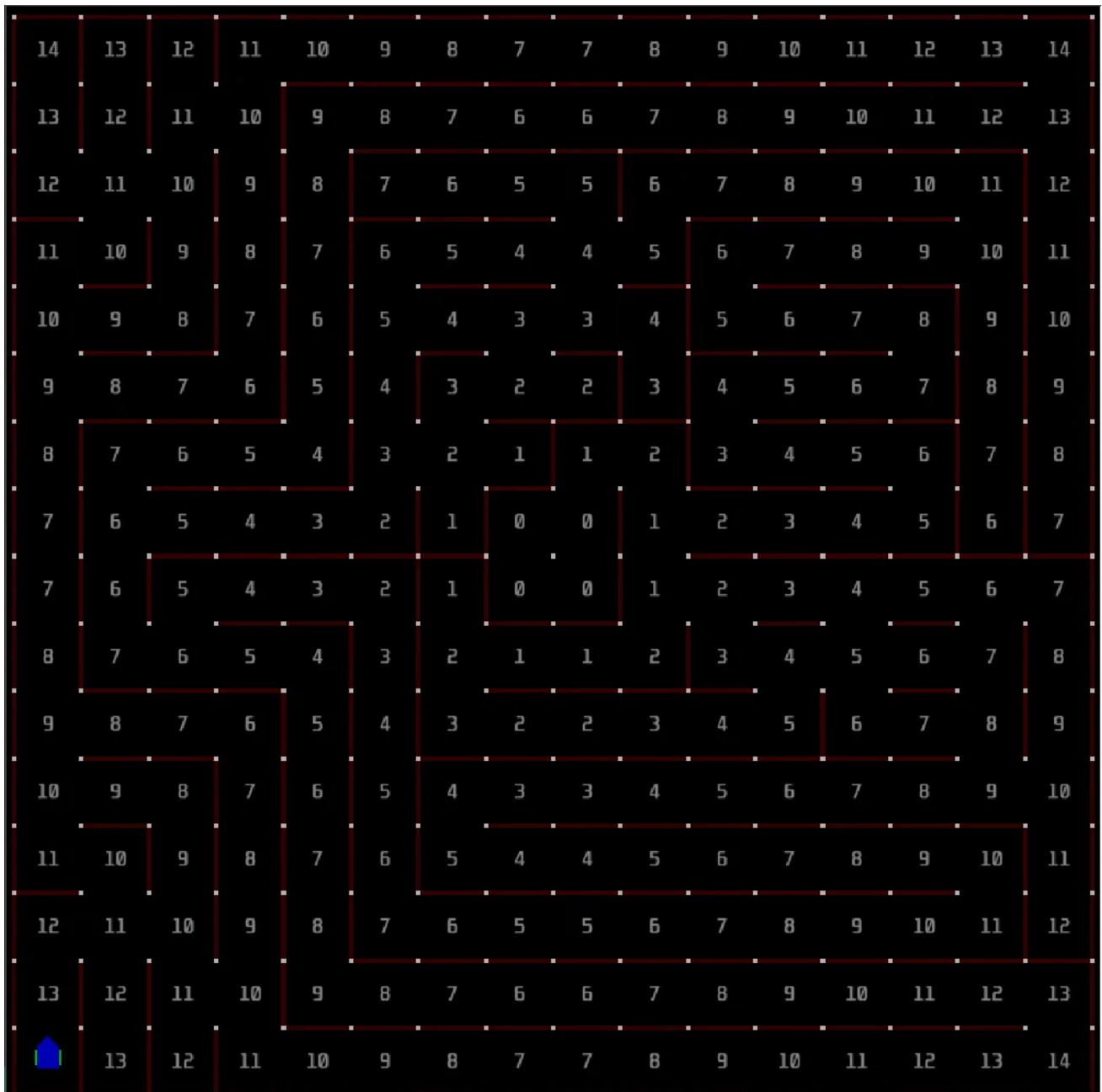
which is immediately accessible by the destination cells, and so on. The cells to which the water flows last will get the highest number.



If you place your mouse anywhere in the maze, and ask it to travel to the cell with the value 1 less than the value of the cell it is in, the mouse is guaranteed to eventually make it to the destination in the path with minimum number of cells.

The traversal

But since your mouse isn't aware of the walls configuration when it is first kept on the maze, first it assumes that there are no walls at all. (except the walls at the maze boundaries). This is what your **Flood array** looks like.



As the mouse continues moves through the maze,

1. Finds the values of it's neighboring cells (from the flood array)
2. Travels to the neighboring cell with the least value.
3. Detects the walls to its left, right and the front
4. Updates the newly found walls in the **maze array**
5. Perform the flood fill for the entire **flood array**
6. Back to step 1, and continue until the robot moves to the desired position.

Observe the flood array when the robot is halfway to the destination. This is because the mouse has discovered the walls in some cells, but not in others. (figure 1)

If you want to send the mouse to some other cell (not to one of the middle four), all you have to do is, perform the floodfill from your desired cell. ie, start by putting zero to the cell you expect the mouse to travel to. (figure 2)

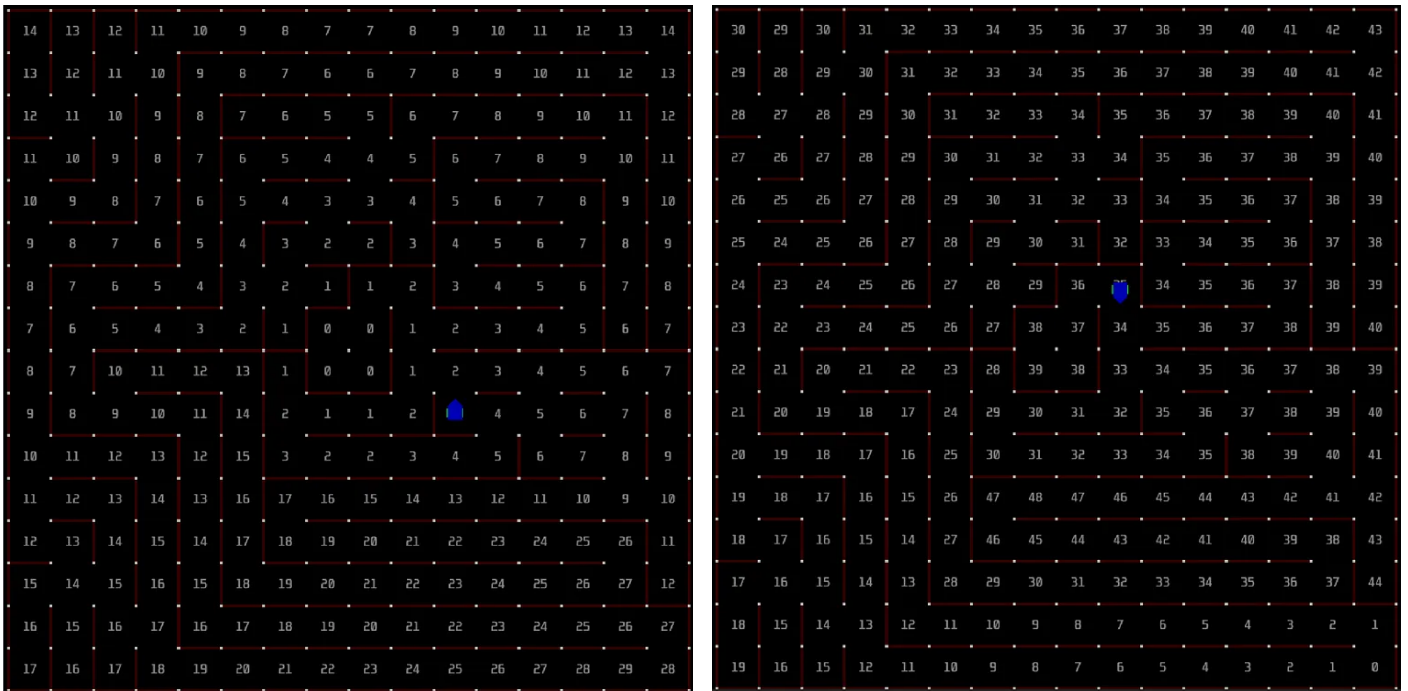


Figure 1 — Left, Figure 2- Right

The fast run

Once you decide that the mouse has discovered enough cells to find an optimum path, you can bring the mouse back to the starting square, and do the fast run. In the process, the mouse

1. Finds the values of it's neighboring cells (from the flood array)
2. Travels to the neighboring cell with the value 1 less than the present cell.
3. Back to step 1, and continue until the robot moves to the desired position.

During the fast run, we don't need to update the **maze array** or the **flood array** as the mouse will only be moving to the cells that are already discovered.

Take a look at our [code](#) to clear out any confusions. Also, take a look at the next article of the series, Testing and Debugging.

Best of luck with your micromouse!!